In the last couple of chapters, instead, we had focused on abstract graphs and finding meaningful ways to embed them in the plane.

If you recall when we presented simulated annealing, we said it can work well with discrete cost functions, and even step-shaped cost functions: those, as we have seen, are some of the situations where it’s advisable to prefer simulated annealing over gradient descent.

Therefore, how could we close this chapter without trying to use simulated annealing to crack the *minimum edge crossing* problem?

17.3.1 Minimum Edge Crossing

As before, to apply our template for simulated annealing, shown in listing 17.1, to a concrete problem, we need to specify two functions: the cost function, and the update step.

The former is often implied by the problem, and so it is here, for the basic version of the problem: we just count how many edges intersect.

The latter, the update step, leaves more leeway to the algorithm designer: should we just move a single vertex randomly? By how much? Should we swap the positions of two vertices? Should edges be kept into consideration during update?

The one advice I can give in these situations is to start small, get something simple working, and then try to add new ideas and validate them by measuring if they help convergence or